# Конструктивная генерация решаемых конфигураций пятнашек с компактной программой физических ходов

Поля размером 1 000 000 × 1 000 000 и адресный доступ к неявной перестановке

Quendoris

14 октября 2026 г. · Редакция для научного рецензирования


## Аннотация

Представлен алгоритм построения физически достижимых конфигураций пятнашек, определяемых компактной последовательностью допустимых перемещений пустой клетки. Последовательность формируется из параметризованных маршрутов на прямоугольных областях и представляется деревом повторений и композиций. Конечная конфигурация вычисляется адресно по композиции обратимых перестановок позиций; точное решение строится обращением программы перемешивания. Рассмотрена реализация для квадратной доски со стороной 1 000 000 и $10^{12}$ позициями. Для фиксированного профиля из 384 раундов и трёх детерминированных seed получены сериализованные JSON-артефакты размером от 678 347 до 678 399 байт. Артефакты описывают решения длиной порядка $10^{26}$ физических ходов без их развёртывания в памяти. На малых досках корректность проверялась полным исполнением последовательностей; на миллионной стороне — адресными и структурными тестами. Приведены формулировка корректности, правила адресного чтения и воспроизводимый протокол проверки.


**Ключевые слова:** пятнашки; композиция перестановок; гамильтонов цикл; символическая программа; рандомизированная конструкция; адресный доступ; обратимость.

**Автор и лицензия:** © 2026 Quendoris. Текст и оригинальные иллюстрации распространяются на условиях Creative Commons Attribution 4.0 International (CC BY 4.0). При повторном использовании необходимо указать автора, источник, лицензию и внесённые изменения. Первоначальная опубликованная редакция будет доступна по адресу quendoris/papers — constructive-million/v1.0.0.

## 1. Определения

Пусть $s$ — чётное целое число $s \geq 4$, $N = s^2$. Квадратная доска содержит различимые фишки $1, \dots, N-1$ и пустую клетку $0$. Каноническая конфигурация в строковой развёртке:

$$B_0 = [1, 2, \dots, N-1, 0].$$

**Физический ход** $U, D, L, R$ — перестановка пустой клетки с соседней по стороне фишкой, если соседняя клетка существует. Направление относится к движению *пустой клетки*. Любая последовательность, в которой каждое действие допустимо для текущего положения пустой клетки, называется допустимой программой ходов.

Для $s = 10^6$ поле содержит $10^{12}$ позиций. Номера фишек требуют 40 бит на элемент. Следовательно, плотный массив с идеальной фиксированной 40-битной упаковкой занимает $5 \cdot 10^{12}$ байт (5 ТБ), а 64-битный массив — $8 \cdot 10^{12}$ байт. Эти величины задают размеры выбранных *развёрнутых* представлений. Они не являются информационной нижней границей для специально конструируемого семейства конфигураций.

Задача состоит в построении по $(s, \sigma, \theta)$, где $\sigma$ — seed, а $\theta$ — фиксированный профиль генератора, пары:

$$(B_\sigma,\ S_\sigma),$$

где $B_\sigma$ адресуемо без хранения всех клеток, а $S_\sigma$ — конечная программа допустимых физических ходов, переводящая $B_\sigma$ в $B_0$.

## 2. Конструкция

Алгоритм строит программу перемешивания $Q_\sigma$ из допустимых ходов пустой клетки; её исполнение определяет

$$B_\sigma = Q_\sigma(B_0), \qquad S_\sigma = Q_\sigma^{-1}.$$

В исследуемой реализации программа содержит прямолинейные переходы пустой клетки и обходы прямоугольных областей по специальным гамильтоновым циклам. Для каждого цикла задаются размеры и ориентация прямоугольника, положение его опорного угла и число повторений. У используемого гамильтонова цикла есть чётная сторона прямоугольника; реализованная версия строит маршруты внутри доски с сохранением непрерывной траектории пустой клетки. Последовательность раундов выбирается детерминированно из $\sigma$ и параметров профиля. В экспериментальном профиле используется режим `wandering`, в котором прямоугольники перемещаются по доске.

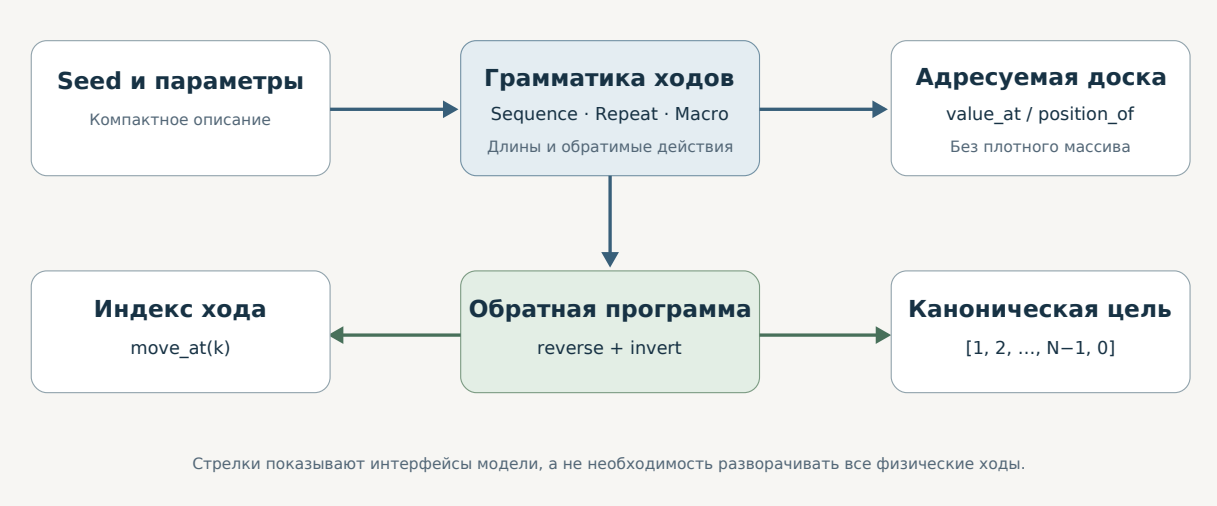



**Рис. 1:** Построение конфигурации и обратной программы

Полное перечисление элементарных ходов при построении $Q_\sigma$ не требуется: примитивы задают ту же последовательность символически, а преобразования координат позволяют вычислять действие готового макроса на отдельную позицию.

## 3. Лемма об обратимости

Пусть $Q = q_1 q_2 \dots q_L$ — допустимая программа из канонической конфигурации, а $\bar{U} = D$, $\bar{D} = U$, $\bar{L} = R$, $\bar{R} = L$.

**Лемма 1.** Программа $Q^{-1} = \bar{q}_L \dots \bar{q}_2 \bar{q}_1$ состоит из допустимых ходов, причём $Q^{-1}(Q(B_0)) = B_0$.

**Доказательство.** Каждый ход меняет местами пустую клетку и одну соседнюю фишку. Обратный шаг из полученной позиции допустим и восстанавливает состояние до обмена. Последовательное применение обратных действий в обратном порядке возвращает все промежуточные состояния, заканчивая $B_0$. □

**Следствие 1.** Для всякого значения seed, при котором конструктор выдаёт корректную последовательность $Q_\sigma$, полученная конфигурация принадлежит компоненте достижимости $B_0$, а $S_\sigma$ является её точным физическим решением.

Это утверждение не зависит от числа клеток или числа ходов. Для программного приложения необходимо отдельно установить, что реализация каждого параметризованного примитива порождает последовательность легальных соседних шагов и что состыкованные примитивы согласуют положение пустой клетки. Для прямолинейных ходов это следует непосредственно из смежности клеток. Для гамильтоновых макропрограмм с хотя бы одной чётной стороной прямоугольника построен независимый комбинаторный разбор: строковый змееобразный обход посещает каждую клетку ровно один раз и замыкается через первый столбец, а вариант с чётной шириной получается транспонированием. Формулировка и проверка геометрических условий вынесены в `docs/proofs/hamiltonian-rectangle-cycle.md` публичного пакета.

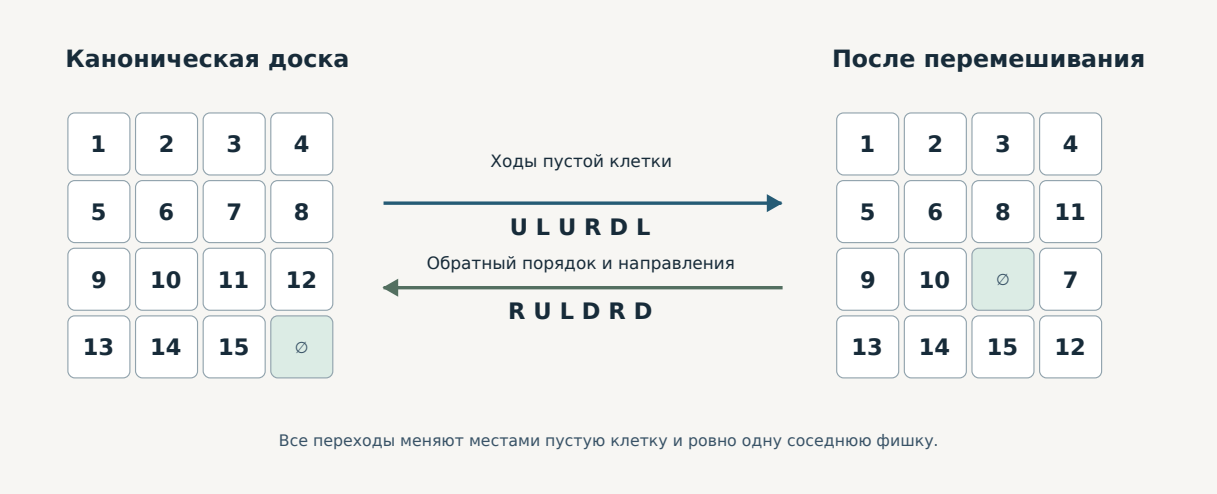



**Рис. 2:** Демонстрация полного исполнения и обращения на доске 4×4

## 4. Исполняемая грамматика

В реализации `recursive_plan_grammar.py` используются четыре непустых вида узлов:

```
Run(direction, count)
PatternRepeat(pattern, count)
Sequence(children)
Repeat(child, count)
```

Для $\mathrm{Run}(d, n)$ длина равна $n$, а $\mathrm{move_at}(k) = d$. Для $\mathrm{PatternRepeat}(p, n)$ длина равна $|p|n$, а ход определяется символом $p[k \bmod |p|]$. Для $\mathrm{Repeat}(A, n)$ индекс передаётся в $A$ как $k \bmod |A|$. Для $\mathrm{Sequence}(A_1, \dots, A_m)$ поддерживается таблица префиксных длин $\ell_j = \sum_{i \le j} |A_i|$; бинарный поиск находит дочерний узел, содержащий заданный индекс.

Таким образом, $\mathrm{move_at}(k)$ не требует развёртывания ни предшествующих ходов, ни всей программы. Сложность зависит от глубины дерева, стоимости поиска по дочерним узлам и адресного правила примитива. Претензия на универсальное $O(1)$ для произвольного дерева здесь не заявляется.

Обращение дерева определяется структурно:

$$\mathrm{inv}(\mathrm{Seq}(A_1, \dots, A_m)) = \mathrm{Seq}(\mathrm{inv}(A_m), \dots, \mathrm{inv}(A_1)),$$
$$\mathrm{inv}(\mathrm{Repeat}(A, n)) = \mathrm{Repeat}(\mathrm{inv}(A), n).$$

Внутри `Run` меняется направление, а внутри `PatternRepeat` — порядок и направление символов шаблона. Длина обратного дерева совпадает с длиной исходного.

Реализация использует целые числа Python произвольной точности для счётчиков ходов

и индексов. Ограничение 64-битных целых к ним не применяется; возможные ограничения стандартного десятичного преобразования очень длинных чисел относятся к форматированию, а не к арифметике алгоритма.

## 5. Неявная конфигурация и адресные запросы

Для каждой макрооперации поддерживаются прямое преобразование координат $T_i$ и обратное $T_i^{-1}$. Их композиция $T = T_m \circ \cdots \circ T_1$ является перестановкой позиций доски. Тогда

$$\text{value_at}(p) = B_0(T^{-1}(p)),$$

а для идентификатора фишки $v$

$$\text{position_of}(v) = T(\text{canonical_position}(v)).$$

Эти два интерфейса должны удовлетворять проверяемому тождеству

$$\text{position_of}(\text{value_at}(p)) = p.$$

В проекте `seeded_legal_hamiltonian_board.py` для запроса обходит список примитивов и вызывает соответствующие позиционные карты, а не имитирует последовательность ходов. Длина этого списка в зафиксированном профиле остаётся конечной и составляет 1 154 для исследованных миллионных конфигураций.

## 6. Зафиксированный профиль и сериализация

Профиль генератора имеет параметры:

| Параметр | Значение |
|---|---|
| Имя | `M9g8-wandering-384` |
| Сторона поля | `1 000 000` |
| Число клеток | `1 000 000 000 000` |
| Режим | `wandering` |
| Число раундов | `384` |
| Примитивы | `1 154` |
| Узлы грамматики | `5 379` |
| Глубина грамматики | `6` |

Файл `million_solution_artifact.py` сериализует параметры профиля, seed, координаты конечной пустой клетки, дерево решения, длину потока, структурные метрики и SHA-256 содержимого. Проверяющая процедура заново строит дерево из профиля и seed, сравнивает полную структуру и длину, затем проверяет хеши и метрики. Файл представляет точный экземпляр данного *конструктивного* семейства задач; он не содержит плотную таблицу перестановки.

Для трёх исследованных seed измерены следующие размеры JSON-файлов и длины логически представленных решений:

| Seed | Размер JSON, байт | Число физических ходов |
|---|---|---|
| 10 363 137 | 678 347 | 96 288 361 077 053 954 149 714 072 |
| 10 363 138 | 678 399 | 95 890 019 446 798 641 740 558 714 |
| 10 363 139 | 678 388 | 99 849 193 096 179 617 002 551 433 |

В архивном M9g8 для трёх seed построение миллионной доски занимало 0,01318–0,01348 с на использованной испытательной машине; отдельная сериализация артефакта в M9g9

занимала 0,0245–0,0272 с. Эти значения относятся к созданию компактного представления, без перечисления клеток и без воспроизведения логического потока физических ходов. Они не характеризуют время исполнения $10^{26}$ перемещений.

Для первого seed соотношение между 5 ТБ плотного 40-битного представления и 678 347 байт файла составляет примерно 7,37 млн к одному. Это отношение размеров *двух конкретных способов представления созданного экземпляра*, не универсальный коэффициент сжатия произвольных перестановок. Оно также не является сравнением производительности решения произвольного входного состояния.

Контрольная сумма SHA-256 учитывает сериализованные данные; внешнее происхождение кода и файла подтверждается отдельно — например, подписью выпуска и зафиксированным коммитом. Сам хеш без доверенного эталона не подтверждает авторство.

## 7. Проверки корректности

### 7.1. Полное физическое исполнение на малых полях

В тесте M9g0 на 20×20 воспроизведены 866 170 ходов перемешивания и столько же обратных ходов. Материализованное конечное состояние совпало с адресным представлением; повторное исполнение обратной последовательности вернуло канонический порядок. В тесте M9g8 с профилем 384 раунда на 8×8 буквально воспроизведены 389 760 ходов в каждом направлении. Проверялись взаимно обратные запросы фишек и позиций.

### 7.2. Адресная верификация большой конфигурации

M9g8 проверил три seed. Для каждого зафиксированы 256 взаимно-обратных запросов по позициям, 256 по фишкам и 261 адресная проверка соответствия ходов обратной программы исходной. Эти тесты не материализуют триллион клеток и не перечисляют физический поток.

M9g9 проверил детерминированную сериализацию и восстановление трёх артефактов, эквивалентность повторно построенной структуры и обнаружение изменения данных по SHA-256. Тест M9g10 для первого seed проверил шесть обращений к произвольным клеткам/фишкам, шесть отдельных ходов, три диапазона общей длиной 1 792 хода, а также защиту от выхода за границы диапазона.

### 7.3. Статистическая характеристика семейства

Предшествующие испытания M9g1–M9g7 исследовали семейства обходов с различной геометрией и числом раундов, используя фиксированные seed, стратифицированные адресные выборки и контрольные распределения. По результатам M9g7 для нового зафиксированного профиля выбрано 384 раунда. Проверенная физическая достижимость не подразумевает статистической равномерности среди всех разрешимых перестановок; данные о качестве перемешивания характеризуют только указанные измеренные метрики.

## 8. Область применимости

Конструкция решает **созданный собственным генератором экземпляр**, поскольку сама программа создания однозначно задаёт обратное физическое решение. Данная статья не утверждает, что произвольная внешняя перестановка может быть решена путём поиска её короткого seed. Она также не утверждает оптимальность числа ходов или равномерность распределения по компоненте достижимости.

Исходная реализация профиля требует чётную сторону доски $s \geq 4$; частные расширения на прямоугольные и нечётные размеры относятся к отдельным вариантам и не входят в приведённый здесь результат.

## 9. Воспроизводимость

Первоначальная исследовательская реализация зафиксирована в `quendoris/uni` на коммите `901933f0f7565b41f0a4f60bf56df5533dc9d65c`. Её публично выделенное ядро находится в репозитории quendoris/implicit-sliding-puzzles, коммит `870adaf0886f85e7dd57e419df9fc48c6944f938` (кандидат PR #1). В публичном пакете доступны конструктор, грамматика, сериализатор, CLI адресных запросов, тесты физического исполнения малых полей и отдельный проверяющий модуль грамматики, не импортирующий конструктор.

Проверки пакета на Python 3.11, 3.12 и 3.13 завершились успешно в GitHub Actions. Контракт verifier ограничен целостностью и семантикой грамматики; он не является самостоятельным формальным доказательством легальности всех параметризованных геометрических макросов. Результаты M9g0, M9g7–M9g10 относятся к исходным архивным экспериментам. Их численные характеристики приведены отдельно от результатов CI нового пакета.

Минимальный повторяемый сценарий для публичного ядра (из корня его checkout):

```
export PYTHONPATH=src
python3 -m unittest discover -s tests -v
python3 -m implicit_sliding_puzzles.million_solution_artifact \
  build --seed 10363137 --output million.solution.json
python3 -m implicit_sliding_puzzles.million_solution_artifact \
  verify million.solution.json
SHA=a8c80310b66d5cc5a8638485a611c4630e09edaf105e4df1ed7807b6fca1dd85
python3 tools/verify_artifact.py million.solution.json \
  --expected-sha256 "$SHA" --indices 0,1,123456789
python3 -m implicit_sliding_puzzles.million_solution_query \
  move-at million.solution.json 123456789
```

Эти команды проверяют сериализованное представление и адресный запрос. Полноту разрешённых геометрических примитивов необходимо оценивать также по математическим определениям и физическим тестам.

## 10. Заключение

Предложена конструкция, задающая физически достижимую перестановку большого игрового поля через компактную исполняемую программу. Теорема обратимости устанавливает существование корректного обратного решения для любой допустимой программы перемешивания. Реализованный профиль для $10^{12}$ клеток хранит решение в JSON-файлах объёмом порядка 0,68 МБ и предоставляет адресные операции для клеток и ходов. Проведённые испытания разделяют полное физическое исполнение на малых полях и структурную/адресную проверку больших экземпляров. Результат предоставляет самостоятельный метод компактного построения и исследования достижимых конфигураций с точно известной физической траекторией.

---

**Архивные и воспроизводимые источники**

1. Публичное ядро и тесты: точный коммит `870adaf0886f85e7dd57e419df9fc48c6944f938`.
2. Первичная исследовательская реализация: фиксированный коммит `901933f0f7565b41f0a4f60bf56df5533dc9d65c`.
3. Архивные результаты `M9g0.stdout.json`, `M9g7.stdout.json`, `M9g8.stdout.json`, `M9g9.stdout.json`, `M9g10.stdout.json`, хранящиеся в доказательной базе исследования.
4. Проверки публичного пакета: три версии Python, тесты физического исполнения и сериализации.